

Python's datetime Module for Data Engineering

Timestamps, time zones, parsing, and formatting for pipelines and ETL

In data engineering, dates and times are rarely simple. Source systems log in different time zones, APIs return inconsistent string formats, and daylight saving time silently breaks naive comparisons. This guide covers Python's built-in `datetime` module with a focus on the problems that actually show up in pipelines: parsing messy timestamps, converting time zones, working with Unix epochs, generating date ranges for partitioning, and avoiding the naive-vs-aware trap.

Object	Represents	Example
<code>date</code>	Calendar date, no time	<code>date(2026, 9, 7)</code>
<code>time</code>	Time of day, no date	<code>time(14, 30, 0)</code>
<code>datetime</code>	Date + time together	<code>datetime(2026, 9, 7, 14, 30)</code>
<code>timedelta</code>	A duration / difference	<code>timedelta(days=1, hours=3)</code>
<code>timezone</code>	A fixed UTC offset	<code>timezone.utc,</code> <code>timezone(timedelta(hours=2))</code>
<code>ZoneInfo</code>	A named IANA time zone	<code>ZoneInfo("Africa/Cairo")</code>

1. The Basics

1.1 Creating datetime objects

```
from datetime import datetime, date, time, timedelta

now = datetime.now() # current local time (naive)
today = date.today() # just the date
specific = datetime(2026, 9, 7, 14, 30, 0) # year, month, day, hour, min, sec
```

1.2 Naive vs. aware datetimes — the most important concept

This is the single biggest source of bugs in pipelines. A **naive** datetime has no time zone attached — Python has no idea if it means UTC, Cairo time, or anything else. An **aware** datetime carries a time zone and can be compared and converted correctly.

```
from datetime import datetime, timezone

naive = datetime(2026, 9, 7, 14, 30) # no tzinfo -> ambiguous
aware = datetime(2026, 9, 7, 14, 30, tzinfo=timezone.utc) # explicit UTC

print(naive.tzinfo) # None
print(aware.tzinfo) # UTC
```

■ Never mix naive and aware datetimes

Comparing or subtracting a naive datetime and an aware datetime raises `TypeError`. In a pipeline, standardize on aware, UTC datetimes as early as possible — ideally right after ingestion — so every downstream step works with the same assumption.

2. Time Zones

2.1 ZoneInfo — the modern way (Python 3.9+)

Use `zoneinfo.ZoneInfo` with IANA time zone names (e.g. "UTC", "Africa/Cairo", "America/New_York") instead of fixed UTC offsets. Named zones automatically account for daylight saving time; fixed offsets do not.

```
from datetime import datetime
from zoneinfo import ZoneInfo

cairo_time = datetime(2026, 9, 7, 16, 0, tzinfo=ZoneInfo("Africa/Cairo"))
utc_time = cairo_time.astimezone(ZoneInfo("UTC"))
ny_time = cairo_time.astimezone(ZoneInfo("America/New_York"))

print(cairo_time) # 2026-09-07 16:00:00+03:00
print(utc_time) # 2026-09-07 13:00:00+00:00
print(ny_time) # 2026-09-07 09:00:00-04:00
```

2.2 Attaching a time zone to a naive datetime

If you know a naive timestamp actually represents a specific zone (common when reading logs from a known source system), attach it with `.replace()` rather than `.astimezone()` — the latter assumes local system time as the source.

```
naive_from_source = datetime(2026, 9, 7, 16, 0) # known to be Cairo local time

# Correct: declare what zone it already is
localized = naive_from_source.replace(tzinfo=ZoneInfo("Africa/Cairo"))

# Then convert to UTC for storage
utc = localized.astimezone(ZoneInfo("UTC"))
```

2.3 Storing in UTC, displaying in local time

■ Pipeline convention

A common and reliable pattern: ingest and store all timestamps in UTC, then convert to a viewer's local zone only at the presentation layer (dashboard, report, API response). This avoids ambiguity in databases and makes cross-region joins consistent.

```
def to_utc_storage(local_dt, source_zone):
    return local_dt.replace(tzinfo=ZoneInfo(source_zone)).astimezone(ZoneInfo("UTC"))

def to_display(utc_dt, viewer_zone):
    return utc_dt.astimezone(ZoneInfo(viewer_zone))
```

3. Parsing and Formatting Timestamps

3.1 Parsing strings with `strptime`

Source systems rarely agree on a format. `strptime` parses a string into a `datetime` given an explicit format code:

```
from datetime import datetime

d1 = datetime.strptime("2026-09-07", "%Y-%m-%d")
d2 = datetime.strptime("07/09/2026 14:30:00", "%d/%m/%Y %H:%M:%S")
d3 = datetime.strptime("Sep 7, 2026", "%b %d, %Y")
```

Code	Meaning	Example
%Y	4-digit year	2026
%y	2-digit year	26
%m	Month (01-12)	09
%d	Day (01-31)	07
%H	Hour, 24h (00-23)	14
%M	Minute (00-59)	30
%S	Second (00-59)	00
%b / %B	Abbreviated / full month name	Sep / September
%z	UTC offset	+0200
%A	Full weekday name	Monday

3.2 ISO 8601 — the format you actually want in pipelines

ISO 8601 (YYYY-MM-DDTHH:MM:SS±HH:MM) is unambiguous, sortable as a string, and the standard for APIs and most databases. Python parses and produces it directly:

```
from datetime import datetime

# Parsing (Python 3.11+ handles the full spec, including 'Z' for UTC)
dt = datetime.fromisoformat("2026-09-07T14:30:00+02:00")

# On Python < 3.11, replace 'Z' manually first:
raw = "2026-09-07T14:30:00Z"
dt = datetime.fromisoformat(raw.replace("Z", "+00:00"))

# Producing ISO format
iso_string = dt.isoformat() # '2026-09-07T14:30:00+02:00'
```

3.3 Formatting for output with `strftime`

```
dt = datetime(2026, 9, 7, 14, 30)

dt.strftime("%Y-%m-%d") # '2026-09-07'
dt.strftime("%Y-%m-%d %H:%M:%S") # '2026-09-07 14:30:00'
dt.strftime("%A, %B %d, %Y") # 'Monday, September 07, 2026'
dt.strftime("%Y%m%d") # '20260907' (common for partition keys)
```

■ Handling multiple inbound formats

For pipelines ingesting from several sources with inconsistent formats, try a list of known formats in order, or use a robust third-party parser like `dateutil.parser.parse()`, which auto-detects most common formats without an explicit pattern.

```
from dateutil import parser

parser.parse("2026-09-07") # works
parser.parse("07 Sep 2026 14:30") # works
parser.parse("September 7, 2026") # works
```

4. Unix Timestamps (Epoch Time)

Many systems (Kafka, logs, APIs, Spark) store timestamps as seconds or milliseconds since the Unix epoch (1970-01-01 UTC). Converting correctly requires being explicit about time zone.

```
from datetime import datetime, timezone

# Unix timestamp -> aware UTC datetime (correct)
dt = datetime.fromtimestamp(1757254200, tz=timezone.utc)

# Aware datetime -> Unix timestamp
ts = dt.timestamp()

# Milliseconds (common in JS-originated data / Kafka)
ts_ms = 1757254200000
dt_from_ms = datetime.fromtimestamp(ts_ms / 1000, tz=timezone.utc)
```

■ `datetime.fromtimestamp()` without `tz` uses local system time

Calling `datetime.fromtimestamp(ts)` without a `tz` argument converts using the machine's local time zone — which changes behavior depending on where the code runs (a laptop vs. a server vs. a container). Always pass `tz=timezone.utc` explicitly in pipeline code.

5. Date Arithmetic and Ranges

5.1 timedelta for durations

```
from datetime import datetime, timedelta

start = datetime(2026, 9, 7)
end = start + timedelta(days=30)
duration = end - start # timedelta(days=30)

print(duration.days) # 30
print(duration.total_seconds()) # 2592000.0
```

5.2 Generating a date range (common for daily partitions)

```
from datetime import date, timedelta

def daterange(start_date, end_date):
    days = (end_date - start_date).days
    for n in range(days + 1):
        yield start_date + timedelta(days=n)

for d in daterange(date(2026, 9, 1), date(2026, 9, 7)):
    print(d.strftime("%Y-%m-%d")) # one partition folder per day
```

5.3 Truncating to day / hour / month boundaries

Useful for bucketing events, deduplicating by day, or building rollups:

```
from datetime import datetime

dt = datetime(2026, 9, 7, 14, 37, 22)

start_of_day = dt.replace(hour=0, minute=0, second=0, microsecond=0)
start_of_hour = dt.replace(minute=0, second=0, microsecond=0)
start_of_month = dt.replace(day=1, hour=0, minute=0, second=0, microsecond=0)
```

6. Dates at Scale — pandas

For DataFrames, pandas builds on datetime but vectorizes everything, which matters once you're past a handful of rows.

```
import pandas as pd

df = pd.read_csv("events.csv", parse_dates=["event_time"])

# Or convert an existing column
df["event_time"] = pd.to_datetime(df["event_time"], utc=True)

# Extract parts
df["date"] = df["event_time"].dt.date
df["hour"] = df["event_time"].dt.hour
df["weekday"] = df["event_time"].dt.day_name()

# Convert time zone
df["local_time"] = df["event_time"].dt.tz_convert("Africa/Cairo")

# Resample into daily buckets (e.g. daily counts)
daily_counts = df.set_index("event_time").resample("D").size()
```

■ `pd.to_datetime(..., utc=True)`

Passing `utc=True` forces every parsed timestamp to be timezone-aware in UTC, even if the source column mixes naive and offset-aware strings. This avoids a common pandas pitfall where a column silently becomes object dtype instead of `datetime64` because of inconsistent inputs.

7. Common Pitfalls in Data Engineering

- **Naive datetimes in a distributed system:** a naive timestamp means nothing across services running in different regions. Standardize on aware UTC datetimes at ingestion.
- **`datetime.now()` vs `datetime.utcnow()`:** `datetime.utcnow()` is deprecated and returns a naive datetime that only looks like UTC — prefer `datetime.now(timezone.utc)`, which is aware and correct.
- **Daylight saving time:** fixed UTC offsets (`timezone(timedelta(hours=2))`) don't shift with DST. Use `ZoneInfo` named zones for anything user-facing or spanning DST transitions.
- **Ambiguous local times:** the hour that repeats during a fall-back DST transition can map to two different UTC times. If this matters, capture the UTC offset from the source rather than reconstructing it later.
- **String-sorting non-ISO dates:** formats like `MM/DD/YYYY` don't sort correctly as strings. Store or partition using ISO 8601 or explicit `YYYYMMDD` keys instead.
- **Mixing seconds and milliseconds epoch values:** always confirm the unit before converting — dividing or multiplying by 1000 incorrectly silently produces a wildly wrong date (e.g. year 1970 or year 50000+).
- **Month/day ambiguity when parsing:** `"03/04/2026"` means March 4 in the US and April 3 in most of Europe. Confirm the source system's convention rather than guessing.

Task	Recommended approach
Ingest timestamp from any source	Parse, then immediately localize/convert to aware UTC
Store in a database	Aware UTC datetime or ISO 8601 string
Partition files by day	YYYYMMDD or ISO date string keys
Display to a user	Convert UTC to the user's <code>ZoneInfo</code> zone at render time
Compare timestamps across services	Ensure both are aware and in the same zone (UTC)
Bulk column operations	pandas with <code>pd.to_datetime(..., utc=True)</code>